Uma **thread** (ou "fio de execução") em Java é a menor unidade de processamento que pode ser executada por um programa. [1]

Em termos simples, ela representa um caminho sequencial de instruções rodando dentro de um processo. Quando você inicia um programa em Java, ele roda por padrão em uma thread principal (chamada de `main`). Com o uso de threads, o programa consegue realizar **várias tarefas ao mesmo tempo** (paralelismo ou concorrência). [1, 2, 3, 4, 5]

Principais conceitos

- **Compartilhamento de memória:** As threads de um mesmo processo compartilham a mesma memória heap e os recursos do programa. Isso facilita a troca de dados, mas exige cuidado para evitar conflitos. [1]
- **Multithreading:** É a capacidade que o Java possui de gerenciar várias threads executando simultaneamente na Java Virtual Machine (JVM). [1]
- **Uso comum:** São muito usadas para rodar tarefas em segundo plano — como fazer requisições de rede, ler arquivos pesados ou processar cálculos — sem travar a interface ou o fluxo principal do sistema. [1]

Como criar uma thread em Java

Existem duas formas principais de criar uma thread na linguagem: [1]

1. **Estendendo a classe `Thread`:** Você cria uma nova classe que herda de `java.lang.Thread` e sobrescreve o método `run()`.
2. **Implementando a interface `Runnable`:** Você implementa a interface `java.lang.Runnable` (definindo a tarefa dentro do método `run()`) e passa essa instância para um objeto `Thread`. *(Esta é a forma mais recomendada).* [1, 2]

Sincronização

Como várias threads compartilham a mesma memória, duas delas podem tentar alterar a mesma variável ao mesmo tempo, causando erros (condição de corrida). Para resolver isso, o Java utiliza **monitores** e blocos `synchronized` para garantir que apenas uma thread acesse um recurso crítico por vez. [1]